FIBONACCI SERIES
IN DATA STRUCTURES

A Case Study on Algorithms, Dynamic Programming, and Customer Loyalty Rewards

COURSE / UNIT: Data Structures

ASSIGNMENT: Case Study Assignment

SUBMISSION DATE: 16 October 2026

GROUP MEMBERS

1. Obama Kimosop CT206/124757/25
2. Anthony Kipkoech -CT206/124762/25
3. Wambui William Kiarie - CT206/124756/25
4. Mikel Kimutai -CT206/124752/25
5. Stephen Kinyanjui- CT206/702125/25

Declaration and Submission Note

This report presents the design, implementation, complexity analysis, and comparison of recursive, iterative, and dynamic-programming approaches for computing Fibonacci numbers. The Java programs are provided for study and demonstration.

Before submission, the group should confirm the official group size and all registration numbers. The assignment brief specifies groups of four, while the names supplied in the earlier draft list five people. Confirm the approved group list with the lecturer and edit the cover page accordingly.

The performance section deliberately does not invent machine-specific benchmark timings. It explains the expected results and includes a benchmark program that the group can run on its own computer. Record the measured values from that run before presenting them as experimental results.

Table of Contents

· 1. Introduction and Problem Statement
2. Task 1 - Recursive Approach
3. Task 2 — Iterative Approach Using an Array
4. Task 3 - Dynamic Programming
5. Task 4 — Customer Loyalty Reward System
6. Task 5 - Performance Analysis forn=30
7.Comparison and Recommendation
8. Testing, Limitations, and Improvements
9. Conclusion
References
Appendix A - Complete Java Programs

1. Introduction and Problem Statement

1.1 Background

The Fibonacci sequence is a sequence of numbers in which each term after the first two is the sum of the two preceding terms. Using the common computer-science convention,F(0)=0andF(1)=1.Forn≥2,F(n)=F(n-1)+F(n - 2). The sequence begins 0, 1, 1, 2, 3, 5,8, 13, 21, 34, 55,...

Fibonacci numbers are useful for demonstrating recursion, loops, arrays, memoization, tabulation, and the trade-off between time and memory. In this case study, a software company wants to use Fibonacci-based values as an example model for customer loyalty reward points.

1.2 Problem Statement

The company needs a program that can calculate reward values for successive customer loyalty years without wasting processing time. A straightforward recursive solution is easy to understand but repeats calculations. More efficient approaches should reuse previously computed values and should be suitable for many customers.

1.3 Objectives

- Implement a recursive function that returns the nth Fibonacci number and explain its efficiency.
- Implement an iterative solution that stores intermediate values in an array and compare it with recursion.
- Use dynamic programming through memoization and tabulation, explaining time and space trade-offs.
- Create a simple reward-point application that accepts a number of years and displays the Fibonacci-based points for each year.
- Compare the approaches forn=30and recommend an appropriate solution for a scalable system.

1.4 Assumptions and Indexing

The report uses zero-based Fibonacci indexing: year/index 0 maps to 0 points, year/index 1 maps to 1 point, and each later index maps to the sum of the previous two values. A real loyalty program would need business rules such as a starting-year convention, maximum points, expiry, and fraud prevention. Those rules are outside this mathematical demonstration.

1.5 Mathematical Definition

F(θ)=θ
F(1)=1
F(n)=F(n-1)+F(n-2),forn>=2

n	F(n)	n	F(n)
0	0	6	8
1	1	7	13
2	1	8	21
3	2	9	34
4	3	10	55
5	5	11	89

2. Task 1 — Recursive Approach

2.1 Explanation

Recursion occurs when a method calls itself to solve smaller instances of the same problem. The recursive Fibonacci method has two base cases,n=0andn=1.. For any larger n, it returns fib(n-1) + fib(n-2)

2.2 Java Implementation

```
public class RecursiveFibonacci {
    public static long fib(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("n must be non-negative");
        }
        if (n == 0 || n == 1) {
            return n;
        }
        return fib(n - 1) + fib(n - 2);
    }

    public static void main(String[] args) {
        int n = 10;
        System.out.println("F(" + n + ") = " + fib(n));
    }
}
```

2.3 Sample Output

```
F(10) = 55
```

2.4 How It Works

To calculate F(5), the method calculatesF(4)+F(3). F(4) in turn calculatesF(3+F(2, while F(3) is calculated again in another branch. These repeated calls are overlapping subproblems.

```
fib(5)
= fib(4) + fib(3)
= (fib(3) + fib(2)) + (fib(2) + fib(1))
= 5
```

2.5 Complexity Analysis

· Time complexity: exponential. A common upper-bound description is O(2^n); a tighter asymptotic description 5 $\Theta(\phi^{\wedge}n)$, where$\phi \approx 1.618$. The method performs many repeated calls.

· Auxiliary space: O(n) due to the maximum recursion depth and call stack.

· Correctness: the base cases match F(0) and F(1), and the recursive rule directly implements the Fibonacci recurrence.

2.6 Advantages and Disadvantages

Advantages	Disadvantages
Short and closely follows the mathematical definition.	Repeats the same subproblems many times.
Useful for teaching recursion and base cases.	Becomes slow as n grows.
Easy to reason about for small n.	Consumes call-stack space and can eventually risk stack overflow for large n.

Conclusion for Task 1: plain recursion is suitable for demonstrating the sequence and calculating small values, but it is not the preferred approach for a production reward system that must serve many requests.

3. Task 2 — Iterative Approach Using an Array

3.1 Explanation

An iterative solution uses a loop instead of recursive calls. An array stores each Fibonacci value once. Once F(i-1)and F(i - 2) are available, the program calculates F(i) and saves it at array index i.

3.2 Java Implementation

```
import java.util.Arrays;

public class IterativeFibonacci {
    public static long[] sequence(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("n must be non-negative");
        }

        long[] fib = new long[n + 1];
        if (n >= 1) {
            fib[1] = 1;
        }

        for (int i = 2; i <= n; i++) {
            fib[i] = fib[i - 1] + fib[i - 2];
        }
        return fib;
    }

    public static void main(String[] args) {
        int n = 10;
        long[] values = sequence(n);
        System.out.println("Sequence through F(" + n + "):");
        System.out.println(Arrays.toString(values));
    }
}
```

3.3 Sample Output

```
Sequence through F(10):
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

3.4 Dry Run

i	fib[i-2]	fib[i-1]	Calculation	fib[i]
2	0	1	*0 + 1*	1
3	1	1	*1 + 1*	2
4	1	2	*1 + 2*	3
5	2	3	*2 + 3*	5
6	3	5	*3 + 5*	8
7	5	8	*5 + 8*	13

3.5 Complexity Analysis

· Time complexity: O(n), because the loop calculates each value from 2 through n once.

· Space complexity: O(n), because the array containsn+1values.

3.6 Comparison with Recursion

The iterative array method avoids repeated work and has predictable linear running time. It is also useful when the application needs the entire sequence, because all values remain stored. Its main cost is the O(n) memory used by the array. For a single Fibonacci number, a two-variable iterative solution can reduce auxiliary space to O(1), but it does not retain the full sequence.

Feature	Plain recursion	Iterative array
Time	Exponential	O(n)
Extra space	O(n) call stack	O(n) array
Stores all values	No	Yes
Best use	Small examples / teaching	Generating and displaying the series

4. Task 3 — Dynamic Programming

4.1 Overview

Dynamic programming (DP) improves performance by solving each subproblem once and reusing its result.Fibonacci has overlapping subproblems because the same smaller indices are requested repeatedly, and it has optimal substructure because F(n) is defined using smaller Fibonacci values.

4.2 Memoization (Top-Down DP)

Memoization keeps the recursive structure but stores computed results in a cache. Before calculating fib(n), the method checks whether that value is already available.

```
import java.util.Arrays;

public class MemoizedFibonacci {
    public static long fib(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("n must be non-negative");
        }
        long[] memo = new long[n + 1];
        Arrays.fill(memo, -1L);
        memo[0] = 0;
        if (n >= 1) memo[1] = 1;
        return solve(n, memo);
    }

    private static long solve(int n, long[] memo) {
        if (memo[n] != -1L) return memo[n];
        memo[n] = solve(n - 1, memo) + solve(n - 2, memo);
        return memo[n];
    }

    public static void main(String[] args) {
        System.out.println("F(30) = " + fib(30));
    }
}
```

Time complexity: O(n), because each index is computed at most once. Space complexity: O(n) for the memo array and up to O(n) recursion stack. This implementation uses -1 as the uncomputed marker; it is safe because valid Fibonacci values are non-negative.

4.3 Tabulation (Bottom-Up DP)

Tabulation starts with the base values and fills the table from smaller indices to larger ones. It avoids recursive calls.

```
public class TabulatedFibonacci {
    public static long fib(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("n must be non-negative");
        }
        if (n <= 1) return n;

        long[] dp = new long[n + 1];
        dp[0] = 0;
        dp[1] = 1;

        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
    }

    public static void main(String[] args) {
        System.out.println("F(30) = " + fib(30));
```

```
    }
}
```

Time complexity: O(n). Space complexity: O(n). Tabulation is a good choice when the program needs repeated access to several values or must display the commplete series.

4.4 Space-Optimized Dynamic Programming

To compute only F(n), the algorithm does not need to retain the entire table. It only needs the two preceding values.

values.

```
public class OptimizedFibonacci {
    public static long fib(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("n must be non-negative");
        }
        if (n <= 1) return n;

        long previous = 0;
        long current = 1;

        for (int i = 2; i <= n; i++) {
            long next = previous + current;
            previous = current;
            current = next;
        }
        return current;
    }

    public static void main(String[] args) {
        System.out.println("F(30) = " + fib(30));
    }
}
```

Memory-efficient option among the implementations in this report when only one Fibonacci value is required.

4.5 Space-Time Trade-Offs

Approach	Time	Auxiliary space	When to use
Naive recursion	Exponential	O(n) stack	Teaching / very small n
Memoization	O(n)	O(n) cache + stack	Recursive formulation with reuse
Tabulation array	O(n)	O(n)	Need many values or the full series
Space-optimized DP	O(n)	O(1)	Need one value with loW memory

4.6 Numeric Range

Java long can represent Fibonacci values only up to F(92). Larger Fibonacci numbers overflow a signed 64-bit long.If the application must support larger indices, use java.math.BigInteger and check the allowed maximum input.The sample loyalty application below deliberately uses long and rejects indices above 92 so that overflow is not silently presented as a valid result.

5. Task 4 — Customer Loyalty Reward System

5.1 Application Design

The program asks the user how many years of loyalty to display. It stores the points in an array, usingF(0)=0and F(1)=1,then prints the points for every index from year 0 through the requested final year. For this demonstration, the year number is treated as the Fibonacci index.

5.2 Complete Java Program

```
import java.util.Scanner;

public class LoyaltyRewardSystem {
    private static long[] buildRewards(int finalYear) {
        if (finalYear < 0 || finalYear > 92) {
            throw new IllegalArgumentException(
                "Enter a year/index between 0 and 92."
            );
        }

        long[] points = new long[finalYear + 1];
        if (finalYear >= 1) points[1] = 1;

        for (int year = 2; year <= finalYear; year++) {
            points[year] = points[year - 1] + points[year - 2];
        }
        return points;
    }

    public static void main(String[] args) {
        try (Scanner input = new Scanner(System.in)) {
            System.out.print("Enter final loyalty year/index (0-92): ");

            if (!input.hasNextInt()) {
                System.out.println("Invalid input. Enter a whole number.");
                return;
            }

            int finalYear = input.nextInt();
            if (finalYear < 0 || finalYear > 92) {
                System.out.println("Year/index must be between 0 and 92.");
                return;
            }

            long[] points = buildRewards(finalYear);
            System.out.println("\nYear/Index\tReward Points");
            for (int year = 0; year < points.length; year++) {
                System.out.println(year + "\t\t" + points[year]);
            }
        }
    }
}
```

5.3 Sample Run

```
Enter final loyalty year/index (0-92): 10

Year/Index    Reward Points
0             0
1             1
2             1
3             2
4             3
5             5
6             8
7             13
8             21
```

```
9               34
10              55
```

5.4 Algorithm

1. Read the final year/index from the user.
2. Check that the input is an integer from 0 to 92.
3. Create an array with finalYear +1elements.
4. Initialize the first two values: points[0] = 0and,when present, points[1] =1
5. For each year from 2 to finalYear, calculate points[year] = points[year - 1] + points[year - 2].
6. Display every year/index and its reward points.

5.5 Complexity

· Time complexity: O(y), where y is the final year/index, because each array element is calculated once.
· Space complexity: O(y), because the program storesy+1values.

5.6 Scalability and Business Considerations

The O(y) approach is substantially more scalable than naive recursion. However, a real system processing thousands of customers should not recompute the sequence recursively for each customer. It could precompute an approved reward schedule once, cache values, or compute the needed index iteratively. If every customer has the same year-based schedule, the same table can be reused.

A real loyalty system also needs a clear business definition of year 0, account eligibility, transaction validation,concurrent updates, audit logs, privacy safeguards, and a maximum reward policy. Fibonacci growth becomes very large quickly, so a company should not adopt the raw sequence as a financial reward policy without caps and careful business review.

5.7 Edge Cases

Input/condition	Expected handling
0	Display only index 0 with 0 points.
1	Display indices 0 and 1: 0 and 1.
Negative number	Reject it with a clear message.
Text or decimal	Reject it because an integer is required.
Index greater than 92	Reject it to avoid long overflow in this implementation.
Large number of customers	Reuse a precomputed table or calculate each required value with an O(n) method.

6. Task 5- Performance Analysis forn=30

6.1 Correctness Check

UsingF(0)=0andF(1)=1,the correct result isF(30) = 832,040. All implementations should return this value. The naive recursive implementation makes2xF(31)-1=2,692,537 total calls when each invocation, including base-case calls, is counted.

6.2 Theoretical Comparison

Method	Time complexity	Space complexity	Expected behaviour at n = 30
Naive recursion	Exponential (O(2^n) upper bound)	O(n) call stack	Much more work due to repeated calls.
Iterative array	O(n)	O(n)	Computes each value once and stores the series.
Memoization	O(n)	O(n) cache and stack	Each subproblem is solved

			once.
Tabulation	O(n)	O(n)	Predictable loop; stores values.
Space-optimized DP	O(n)	O(1)	Computes only the requested value with constant extra memory.

6.3 Benchmarking Method

Actual elapsed time depends on the computer, Java version, JVM warm-up, background processes, and measurement technique. Do not copy a made-up timing into the report. Run the following benchmark program several times on the presentation computer, record the results, and state the environment used.

6.4 Benchmark Program

```
public class FibonacciBenchmark {
    private static volatile long sink;

    private static long recursive(int n) {
        if (n <= 1) return n;
        return recursive(n - 1) + recursive(n - 2);
    }

    private static long iterativeArray(int n) {
        long[] a = new long[n + 1];
        if (n >= 1) a[1] = 1;
        for (int i = 2; i <= n; i++) a[i] = a[i - 1] + a[i - 2];
        return a[n];
    }

    private static long optimized(int n) {
        if (n <= 1) return n;
        long a = 0, b = 1;
        for (int i = 2; i <= n; i++) {
            long next = a + b;
            a = b;
            b = next;
        }
        return b;
    }

    private static double timeMs(java.util.function.LongSupplier task) {
        long start = System.nanoTime();
        sink = task.getAsLong();
        long end = System.nanoTime();
        return (end - start) / 1_000_000.0;
    }

    public static void main(String[] args) {
        int n = 30;

        // Warm up the JVM using fast methods.
        for (int i = 0; i < 10000; i++) {
            sink = iterativeArray(n);
            sink = optimized(n);
        }

        // Repeat measurements and report the median for each method.
        int runs = 7;
        double[] recursiveTimes = new double[runs];
        double[] arrayTimes = new double[runs];
        double[] optimizedTimes = new double[runs];

        for (int i = 0; i < runs; i++) {
            recursiveTimes[i] = timeMs(() -> recursive(n));
```

```
            arrayTimes[i] = timeMs(() -> iterativeArray(n));
            optimizedTimes[i] = timeMs(() -> optimized(n));
        }

        java.util.Arrays.sort(recursiveTimes);
        java.util.Arrays.sort(arrayTimes);
        java.util.Arrays.sort(optimizedTimes);

        System.out.println("n = " + n);
        System.out.println("F(30) = " + recursive(n));
        System.out.printf("Recursive median: %.6f ms%n", recursiveTimes[runs / 2]);
        System.out.printf("Iterative-array median: %.6f ms%n", arrayTimes[runs / 2]);
        System.out.printf("Space-optimized median: %.6f ms%n", optimizedTimes[runs / 2]);
    }
}
```

6.5 How to Compile and Run

```
javac FibonacciBenchmark.java
java FibonacciBenchmark
```

The benchmark uses System.nanoTime(), performs a short warm-up for the fast methods, repeats each measurement seven times, and reports the median. Very short timings can still be noisy; use multiple program executions and report the general pattern, not false precision. The recursive method atn=30is manageable on typical computers, but do not increase n substantially because its work grows exponentially.

6.6 Results Table to Complete After Running

Method	Median time (ms) from your run	Correct result
Naive recursion	Enter measured value	832040
Iterative array	Enter measured value	832040
Space-optimized iteration	Enter measured value	832040

Expected qualitative result: iterative and dynamic-programming methods should be substantially faster than naive recursion as n increases. The exact measured times must be obtained on the group's own computer.

6.7 Interpretation

Atn=30, naive recursion evaluates millions of calls because it recomputes overlapping subproblems. The iterative array method and dynamic-programming approaches calculate each required Fibonacci value once, producing linear-time behaviour. Space-optimized iteration retains only two previous values, making it the most memory-efficient when the application needs only F(30). When the application must display the full sequence, the array-based method is more convenient because it stores all values.

7. Comparison and Recommendation

Criterion	Naive recursion	Iterative array / tabulation	Space-optimized DP
Efficiency	Poor for larger n	Good: O(n)	Good: O(n)
Memory	Call stack O(n)	Array O(n)	O(1) auxiliary space
Full sequence available	No	Yes	No; only final value
Implementation clarity	Very simple	Simple	Simple to moderate
Recommended use	Teaching small examples	Displaying yearly rewards	Calculating a single value

7.1 Recommendation

For the specified reward-system application, use iterative tabulation with an array. It runs in O(n) time, stores each result once, and makes it straightforward to display the reward points for every year as required by Task 4. If the system only needs a single customer's points for a particular index and does not need the preceding values,

use the space-optimized iterative version to reduce auxiliary memory to O(1). Avoid naive recursion in the production path.

For a service with thousands of customers, precompute the reward schedule up to the allowed maximum once and reuse it, provided the reward rules are identical for all customers. Add validation, an explicit reward cap, tests,and monitoring before production use.

8. Testing, Limitations, and Improvements

8.1 Test Cases

Test	Input	Expected result
Base case	$n = 0$	$F(0) = 0$
Base case	$n = 1$	$F(1) = 1$
Small value	$n = 10$	$F(10) = 55$
Required benchmark	$n = 30$	$F(30) = 832040$
Negative input	$n = -1$	Input rejected
Unsupported range	$n = 93 \in long - basedrewardprogram$	Input rejected
Invalid type	$n = \hbar\ ello$	Input rejected

8.2 Limitations

· The program is a console demonstration, not a complete customer database or web service.

· The use of Fibonacci numbers as reward points is a mathematical case-study assumption, not a recommendation for an actual financial loyalty policy.

· Thelong-basedimplementation is restricted to indices up to 92 to avoid signed 64-bit overflow.

Benchmark timings vary by computer and must be measured locally.

The report does not claimm that the programs have been executed on the lecturer's environment; the group should compile and test each file before presentation.

8.3 Suggested Improvements

· a nit tests for base cases, typical values, invalid inputs, and the maximum supported index.

· Use BigInteger if values beyond F(92) are required

· Separate the rewward calculation logic from user input and display logic.

Store the reward schedule in a database or cache only if the wider applicationrequires persistence.

· Add business controls such as maximum points, eligibility checks, audit trails, and protection against duplicate reward credits.

9. Conclusion

This case study demonstrates how the same Fibonacci problem can be solved using recursion, iteration, and dynamic programming. Naive recursion follows the mathematical definition but repeats many calculations and has exponential running time. Iterative array computation, memoization, and tabulation reduce the time complexity to O(n). Space-optimized iteration also achieves O(n) time while using O(1) auxiliary space when only one value is needed.

For the customer loyalty example, iterative tabulation is recommended because the program must store and display points for each year. A production system serving many customers should reuse a precomputed schedule where possible, validate inputs, define a safe reward policy, and test its implementation. The benchmark should be run on the group's own computer and its actual results inserted before submission.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.Chapter 15: Dynamic Programming.

2. Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Recursion and Dynamic Programming sections.

3. Oracle. Java Documentation. https://docs.oracle.com/en/java/

Arrays and Biglnteger.

4. Kleinberg, J., & Tardos, É. (2006). Algorithm Design. Pearson.

Chapter 6: Dynamic Programming

5. Knuth, D. E. (1997). The Art of Computer Programming, Volume 1: Fundamental Algorithms (3rd ed.). Addison-Wesley.

6. W3Schools. (2024). Java Arrays and Loops. https://www.w3schools.com/java/

7. TutorialsPoint. (2024). Data Structures — Fibonacci Search and Series.
https://www.tutorialspoint.com/data structures algorithms/

Appendix A — Complete Java Programs

Save each public class in a separate file whose filename matches the class name. For example, RecursiveFibonacci must be saved as RecursiveFibonacci.java. Compile each file with javac FileName.java and run it with java ClassName.

A.1 RecursiveFibonacci.java

```
public class RecursiveFibonacci {
    public static long fib(int n) {
        if (n < 0) throw new IllegalArgumentException("n must be non-negative");
        if (n == 0 || n == 1) return n;
        return fib(n - 1) + fib(n - 2);
    }

    public static void main(String[] args) {
        int n = 10;
        System.out.println("F(" + n + ") = " + fib(n));
    }
}
```

A.2 IterativeFibonacci.java

```
import java.util.Arrays;

public class IterativeFibonacci {
    public static long[] sequence(int n) {
        if (n < 0) throw new IllegalArgumentException("n must be non-negative");
        long[] fib = new long[n + 1];
```

```
        if (n >= 1) fib[1] = 1;
        for (int i = 2; i <= n; i++) fib[i] = fib[i - 1] + fib[i - 2];
        return fib;
    }

    public static void main(String[] args) {
        int n = 10;
        System.out.println(Arrays.toString(sequence(n)));
    }
}
```

A.3 MemoizedFibonacci.java

```
import java.util.Arrays;

public class MemoizedFibonacci {
    public static long fib(int n) {
        if (n < 0) throw new IllegalArgumentException("n must be non-negative");
        long[] memo = new long[n + 1];
        Arrays.fill(memo, -1L);
        memo[0] = 0;
        if (n >= 1) memo[1] = 1;
        return solve(n, memo);
    }

    private static long solve(int n, long[] memo) {
        if (memo[n] != -1L) return memo[n];
        memo[n] = solve(n - 1, memo) + solve(n - 2, memo);
        return memo[n];
    }

    public static void main(String[] args) {
        System.out.println("F(30) = " + fib(30));
    }
}
```

A.4 TabulatedFibonacci.java

```
public class TabulatedFibonacci {
    public static long fib(int n) {
        if (n < 0) throw new IllegalArgumentException("n must be non-negative");
        if (n <= 1) return n;
        long[] dp = new long[n + 1];
        dp[0] = 0;
        dp[1] = 1;
        for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
        return dp[n];
    }

    public static void main(String[] args) {
        System.out.println("F(30) = " + fib(30));
    }
}
```

A.5 OptimizedFibonacci.java

```
public class OptimizedFibonacci {
    public static long fib(int n) {
        if (n < 0) throw new IllegalArgumentException("n must be non-negative");
        if (n <= 1) return n;
        long previous = 0, current = 1;
        for (int i = 2; i <= n; i++) {
            long next = previous + current;
            previous = current;
            current = next;
        }
        return current;
    }

    public static void main(String[] args) {
```

```
        System.out.println("F(30) = " + fib(30));
    }
}
```

A.6 LoyaltyRewardSystem.java

```
import java.util.Scanner;

public class LoyaltyRewardSystem {
    private static long[] buildRewards(int finalYear) {
        if (finalYear < 0 || finalYear > 92) {
            throw new IllegalArgumentException("Year/index must be between 0 and 92.");
        }
        long[] points = new long[finalYear + 1];
        if (finalYear >= 1) points[1] = 1;
        for (int year = 2; year <= finalYear; year++) {
            points[year] = points[year - 1] + points[year - 2];
        }
        return points;
    }

    public static void main(String[] args) {
        try (Scanner input = new Scanner(System.in)) {
            System.out.print("Enter final loyalty year/index (0-92): ");
            if (!input.hasNextInt()) {
                System.out.println("Invalid input. Enter a whole number.");
                return;
            }
            int finalYear = input.nextInt();
            if (finalYear < 0 || finalYear > 92) {
                System.out.println("Year/index must be between 0 and 92.");
                return;
            }
            long[] points = buildRewards(finalYear);
            System.out.println("\nYear/Index\tReward Points");
            for (int year = 0; year < points.length; year++) {
                System.out.println(year + "\t\t" + points[year]);
            }
        }
    }
}
```

A.7 FibonacciBenchmark.java

```
public class FibonacciBenchmark {
    private static volatile long sink;

    private static long recursive(int n) {
        if (n <= 1) return n;
        return recursive(n - 1) + recursive(n - 2);
    }

    private static long iterativeArray(int n) {
        long[] a = new long[n + 1];
        if (n >= 1) a[1] = 1;
        for (int i = 2; i <= n; i++) a[i] = a[i - 1] + a[i - 2];
        return a[n];
    }

    private static long optimized(int n) {
        if (n <= 1) return n;
        long a = 0, b = 1;
        for (int i = 2; i <= n; i++) {
            long next = a + b;
            a = b;
            b = next;
        }
        return b;
    }
```

```
    private static double timeMs(java.util.function.LongSupplier task) {
        long start = System.nanoTime();
        sink = task.getAsLong();
        return (System.nanoTime() - start) / 1_000_000.0;
    }

    public static void main(String[] args) {
        int n = 30, runs = 7;
        for (int i = 0; i < 10000; i++) {
            sink = iterativeArray(n);
            sink = optimized(n);
        }
        double[] r = new double[runs], a = new double[runs], o = new double[runs];
        for (int i = 0; i < runs; i++) {
            r[i] = timeMs(() -> recursive(n));
            a[i] = timeMs(() -> iterativeArray(n));
            o[i] = timeMs(() -> optimized(n));
        }
        java.util.Arrays.sort(r);
        java.util.Arrays.sort(a);
        java.util.Arrays.sort(o);
        System.out.println("n = " + n + ", F(30) = " + recursive(n));
        System.out.printf("Recursive median: %.6f ms%n", r[runs / 2]);
        System.out.printf("Iterative-array median: %.6f ms%n", a[runs / 2]);
        System.out.printf("Space-optimized median: %.6f ms%n", o[runs / 2]);
    }
}
```